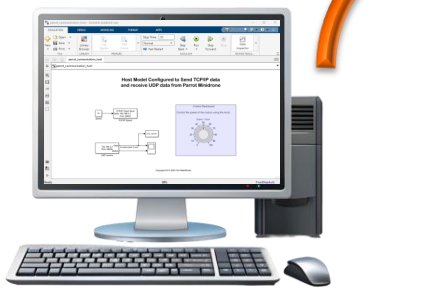
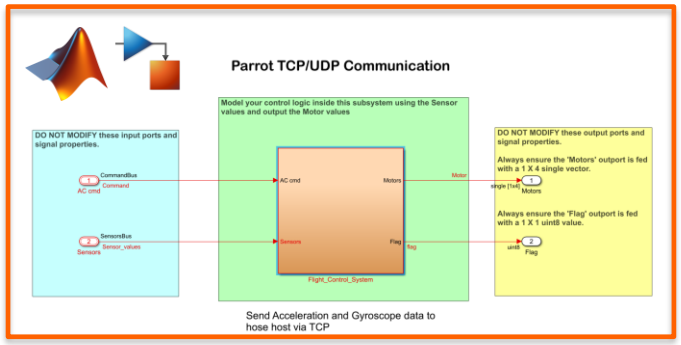
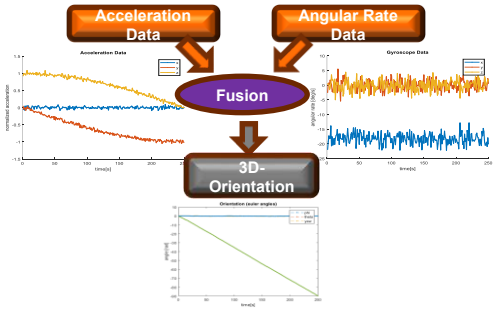
RO2 Project Wintersemester 2025/26

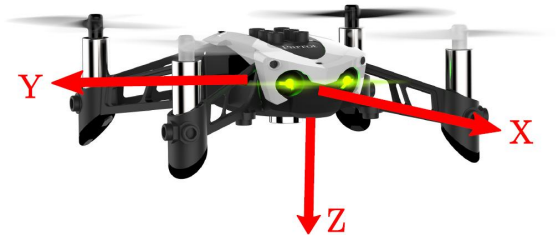
3D-Orientation of a Drone







3D acceleration
3D angular rate



we focus on students

Fachhochschule Dortmund
University of Applied Sciences and Arts

Robotic project

Goal

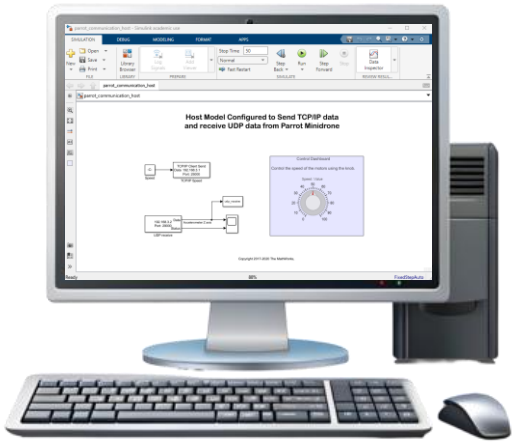
- Determining the 3D-orientation of the Parrot Mambo drone based on the 3D-acceleration and 3D-gyroscope data

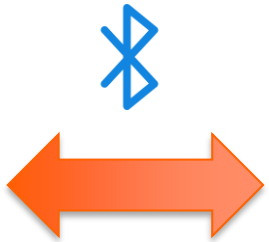
Steps

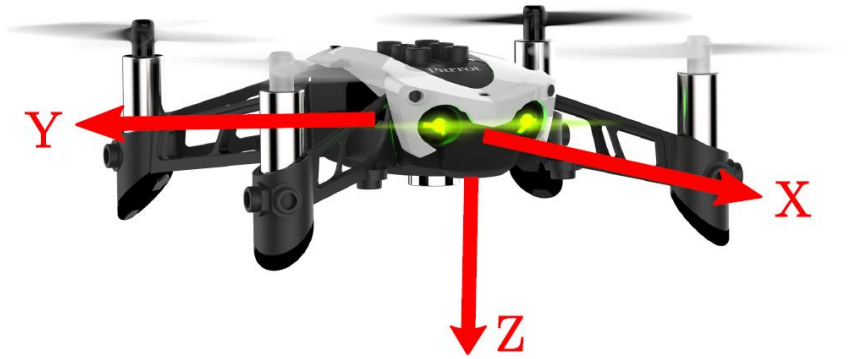
1. Connect the Mambo drone to the PC via Bluetooth
2. Modify the Simulink Controller Model to transmit 3D-acceleration and 3D-gyroscope data
3. Flash the modified Simulink Controller to the Mambo drone
4. Rotate the drone using the end-effector of the Cobotta and capture the drone's sensor data
5. Process the captured sensor data to determine the drone rotations
6. Compare calculated rotation angles to the „real“ rotation angles

1. Connect the Mambo Drone to the PC via Bluetooth

- Follow this instruction to establish a Bluetooth connection between the PC and the Mambo drone







2. Modify the Simulink Controller Model to transmit 3D-acceleration and 3D-gyroscope data

Background

- The communication between the PC and the drone is based on a Simulink **Host Model** running on the PC, and a Simulink **Controller Model** running on the Mambo drone
- Data can be exchanged over Bluetooth either via TCP or UDP
- The Model "**parrot_communication**" described here is an example for a Controller Model which uses TCP to receive the desired motor speeds (sent by the host) and to transmit the *z*-axis data of the drone's accelerometer via UDP (to the host)
- The goal in the next steps is to extend the model to not only sent the *z*-axis data of accelerometer, but also the *x*- and *y*-axis data of the accelerometer as well as the *x*-, *y*-, and *z*-axis data of the gyroscope via UDP
- As we do not want to change the motor speeds, we will also remove the TCP/IP block used for receiving the desired motor speeds from the host

2. Modify the Simulink Controller Model to transmit 3D-acceleration and 3D-gyroscope data (continued)

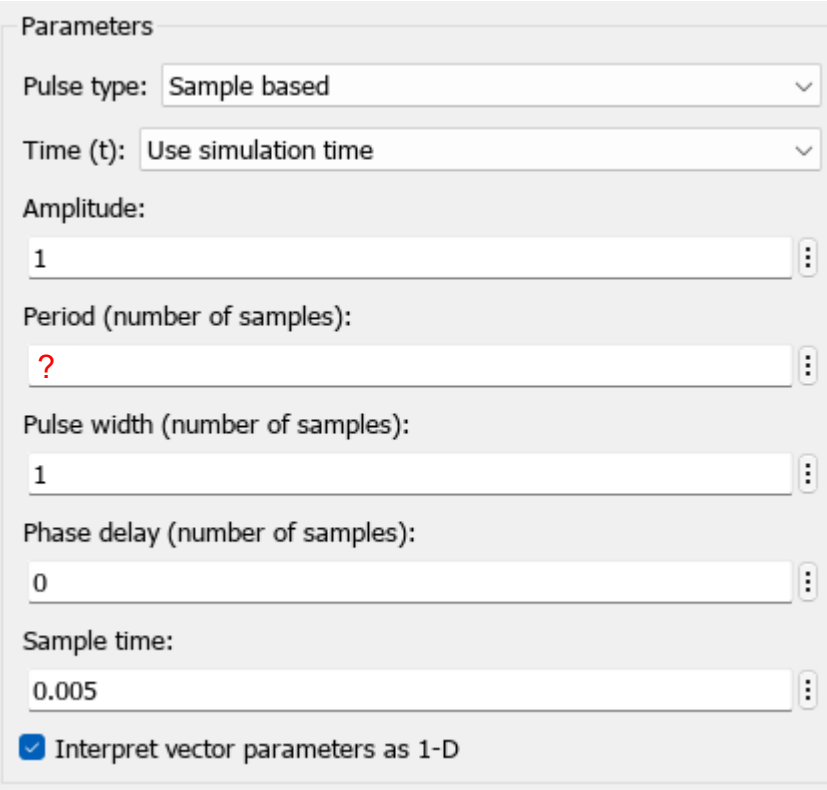
Steps

1. Open the Controller Model “parrot_communication” e.g. by running the following commands in the MATLAB Command window:
   `openExample('parrot/CommunicatingWithPARROTMinidroneTCPUDPExample','supportingFile','parrot_communication')`
2. In the main-subsystem “Flight_Control_System”, remove the `TCP/IP` block labeled “MotorInput” and connect the inputs of the `Vector Concatenate` block used for the Motor Control to the `Constant` block having value 0
3. In the left part of the main-subsystem, the input block labeled “Sensors” is connected to a subsystem; Open that subsystem and identify which of the bus elements represents the *x*-, *y*- and *z*-axis data of the accelerometer and the gyroscope. To which output ports are these elements connected?
4. Back in the main-subsystem “Flight_Control_System”, add a `Vector Concatenate` block to “join”/merge the output ports of the subsystem from the previous step corresponding to the *x*-, *y*- and *z*-axis data of the accelerometer and the gyroscope into a vector and connect the vector output to the input of the triggered subsystem (labeled ‘1’) which is used for UDP transmission

2. Modify the Simulink Controller Model to transmit 3D-acceleration and 3D-gyroscope data (continued)

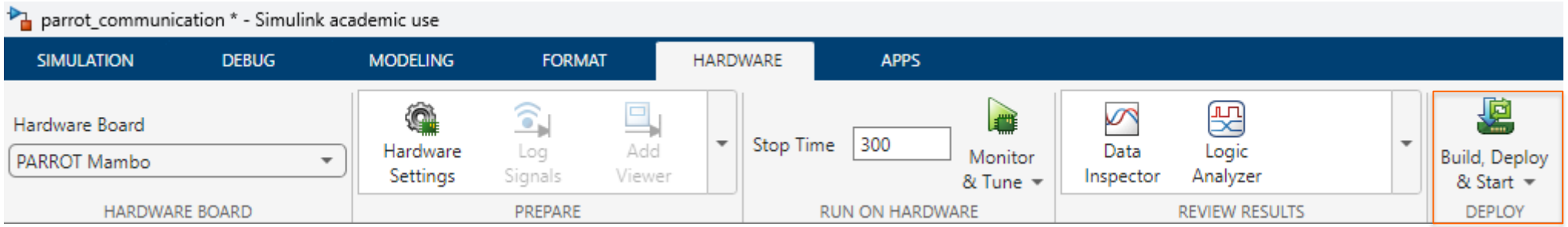
Steps (continued)

5. The UDP transmission rate is realized by a `Pulse Generator` block triggering the `Triggered Subsystem`. Change the parameters of the Pulse Generator to accomplish a transmission rate of 50Hz (s. screenshot on the right)



3. Flash the modified Simulink Controller to the Mambo drone

- In the top panel of the Simulink window, open the Hardware tab
- Set the Stop Time to 300
- Make sure the PC is connected to the drone via Bluetooth, then click on the “Build, Deploy & Start” Button



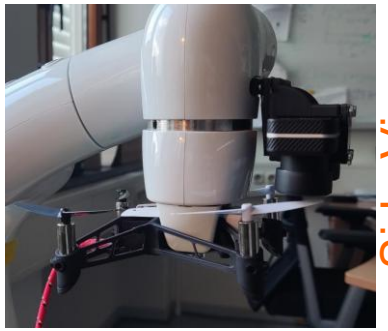
4. Rotate the drone using the end-effector of the Cobotta and capture the drone's sensor data

- The class `RobotClass` can be used to
  - Attach the drone to the end-effector Cobotta (see screenshots on the right). Use the class method `move2measurepos` for this purpose
  - Rotate the end-effector of the Cobotta about *x*, *y* or *z*-axis. Starting with zero-rotation, the sequence is as follows (the given angles indicate the relative change in rotation):

    pause 3s | +45° | pause 3s | +45° | pause 3s | -45° | pause 3s | -45 | pause 3s

  See `Example_RobotClass.m`!

Front View



Side View

- The class `MamboClass` can be used to
  - Connect to a mambo drone and start the controller model running on the drone
  - Run a measurement using the host model and storing the captured sensor data into a matlab file

  See `Example_MamboClass.m`!

4. Rotate the drone using the end-effector of the Cobotta and capture the drone's sensor data (continued

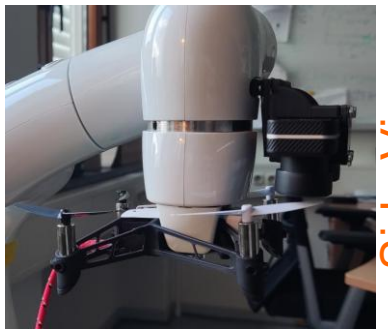
Hints:

- You will have to use both classes in order to *move the end-effector* and *to capture the sensor data* of the mambo drone, e.g.:

```
…
my_mambo.run_measurement('x_axis_rotation', 40);
my_robot.x_axis_rotation();
…
```

- it is beneficial to start a new measurement (using `run_measurement`) for each rotation sequence, so that the measurement data for *x*, *y*, and *z*-rotations are saved into different mat-files
- You can use the `template.m` as starting point
- Make videos of the end-effector/drone rotations for later use

Front View



Side View

Robotic project

Example:

```
my_robot.y_axis_rotation();
```

pause 3s | +45° | pause 3s | +45° | pause 3s | -45° | pause 3s | -45 | pause 3s

5. Process the captured sensor data to determine the drone rotations

For each complete sequence of *x*, *y*, and *z*-rotations:

- Plot the captured raw data (accelerometer and gyroscope) vs. time
- Calculate the rotation angles based only on accelerometer data and plot the result vs. time
- Calculate the rotation angles based only on gyroscope data and plot the result vs. time
- Calculate Quaternions based on gyroscope data, convert these to Euler Angles and plot the result
- Calculate the rotation angles based on sensor fusion using a Complementary Filter and plot the result
- Plot the rotation angles based only on accelerometer and gyroscope and the rotation angles gained by the Complementary Filter into a single figure (3 Subplots)
- Implement a simple offset compensation (for the raw accelerometer and gyroscope data) by determining the offset based on the known initial state of the measurement (no motion, *z*-axis parallel to gravity) and subtracting the offset from all raw data. Repeat the calculations of the rotation angles (previous steps) and plot the results
- Interpret the results (presentation)

6. Compare calculated rotation angles to the „real“ rotation angles

- Based on the Euler Angles determined using the Complementary Filter, determine the angles in the static phases (phases without end-effector movement) e.g. by
  - Marking the phases using MATLAB’s brush tool
  - Exporting the data into variables
  - Calculating the mean of the variables
- Compare the resulting angles with the *desired angles* defined by the rotations of the end-effector, i.e. the absolute static angles

  0 → 45° → 90° → 45° → 0 resulting from the end-effector sequence

➡ **For each angle, calculate the mean error and the standard deviation**